Lavorare con gli agenti di programmazione

Corso di Frontend Development – Modulo 08

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. Lavorare con gli agenti di programmazione
1.1 Cosa sono e cosa fanno
1.2 Il file di contesto del progetto
1.3 Metodo
1.4 Cosa resta vostro

Sommario del modulo

1. Lavorare con gli agenti di programmazione
1.1 Cosa sono e cosa fanno
1.2 Il file di contesto del progetto
1.3 Metodo
1.4 Cosa resta vostro

Dal completamento automatico all'agente

Livello	Cosa fa	Esempi
Completamento	suggerisce la riga successiva	Copilot classico
Chat	risponde e propone codice da incollare	ChatGPT, Claude nel browser
Agente	legge il repo, pianifica, modifica file, *esegue comandi e controlli*, corregge e ripete finché il compito è fatto	Claude Code, Codex, Copilot coding agent, Cursor, Gemini CLI

- L'agente lavora nel **vostro** terminale: apre `frontend/src/pages/CatalogPage.tsx`, lancia `npm run build`, legge gli errori di TypeScript.
- Cambia il vostro ruolo: da chi scrive ogni riga a chi **specifica, indirizza e verifica**.

Dove rende di più

- **Codice meccanico**: le funzioni axios per un gruppo di endpoint (come quelle del carrello in `lib/api.ts`), le colonne di una tabella admin, un form con le stesse regole di validazione delle altre.
- **Traduzioni e migrazioni**: da `useEffect + fetch` sparsi a un hook condiviso in tutte le pagine, da `window.confirm` a `AlertDialog`.
- **Esplorazione**: "come viene aggiunto il Bearer alle richieste in questo repo?", "chi usa `formatPrice`?".
- **Debug**: dato un 401 inatteso dopo otto ore, trovare il punto in cui il token scaduto non viene gestito.
- **Rifiniture**: `aria-label` sui pulsanti con sola icona, messaggi di errore, stati vuoti, documentazione.
- **Lavoro che non avreste fatto**: scrivere `openapi.yaml` per le API di minicommerce partendo dalle rotte vere.

Dove sbaglia (e come)

- **API inventate o vecchie**: "uso `GET /api/brands`" — non esiste. Oppure Tailwind v3 in un progetto con Tailwind v4.
- **Troppo codice**: un `BrandFilterProvider` con context quando serviva un `<Select>`. Chiedete il minimo.
- **Allargamento silenzioso**: "già che c'ero ho riformattato tutto `api.ts`". Il diff va letto tutto.
- **Dipendenze non richieste**: installa un pacchetto per i select quando `components/ui/select.tsx` c'è già.
- **Sicurezza**: segreti in chiaro, CORS aperto lasciato com'è, token spostato in un posto peggiore. La checklist del modulo 6 vale doppio.
- **Sicurezza operativa**: un agente che esegue comandi può lanciare `db:reset` e cancellare il vostro database locale.

minicommerce ha già il suo: AGENTS.md

```
# Repository Guidelines

## Project Structure
backend/src/ holds the Express API, route handlers, auth middleware,
SQLite/Drizzle schema and seed data. frontend/src/ contains the React app,
pages, shared components, API/auth helpers, styles and assets.
The project currently has no dedicated test directory or test runner.

## Build, Test, and Development
Use Node.js 24 or newer. Install dependencies separately in backend/ and
frontend/ with npm install. Backend: npm run dev starts the watch-mode API
on port 3001; npm run db:push applies the Drizzle schema and npm run db:seed
loads demo data and the admin account.
Frontend: npm run dev starts Vite on port 5173
(with /api proxied to the backend). npm run build type-checks and creates
the production bundle; npm run lint runs Oxlint.
npm run db:reset in backend/ deletes and rebuilds the local database,
including reseeding it. Use only when you intend to discard local data.

## Coding Style & Naming
Two-space indentation, semicolons, single quotes in backend files and double
quotes in frontend files. PascalCase for React components, camelCase for
variables/functions. Keep API access in frontend/src/lib/api.ts and database
definitions in backend/src/db/schema.ts.
```

Perché questo file funziona

- **Dice dove stanno le cose**: due cartelle, un ruolo ciascuna. L'agente non va a cercare le rotte dentro `frontend/`.
- **Dà i comandi esatti**: avvio, `db:push`, `db:seed`, build, lint. Senza, l'agente li indovina e sbaglia.
- **Ammette quello che manca**: "no dedicated test runner". Così non vi promette test che non può eseguire.
- **Fissa convenzioni verificabili**: indentazione, punto e virgola, `PascalCase` per i componenti.
- **Indica i confini**: le chiamate HTTP stanno in `lib/api.ts`, lo schema in `db/schema.ts`, non sparsi nelle pagine.
- **Avverte sui comandi distruttivi**: `db:reset` cancella il database. Una riga che vi salva un pomeriggio di dati.

Cosa manca all'`AGENTS.md` di oggi

- – Niente sui **quattro stati** di ogni schermata (caricamento, errore, vuoto, dati): `CatalogPage` ne gestisce tre, l'errore finisce solo in un toast. Un agente non sa che la regola esiste e ne salta un altro.
- – Niente **regole di accessibilità**: nessuno dice "i pulsanti con sola icona vogliono `aria-label`, non `title`" — e nel codice ce ne sono tre fatti così.
- – La regola delle **virgolette** non corrisponde al codice vero: dice apici singoli nel backend e doppi nel frontend, ma `frontend/src/lib/api.ts` usa gli apici singoli.
- – Non dice **cosa non fare**: non committare, non aggiungere dipendenze senza chiedere, non modificare i file copiati in `components/ui/`.
- – Non nomina le **scelte del corso**: un hook per i dati, filtri nell'URL, AlertDialog al posto di window.confirm.
- – Non dice che il progetto è **cresciuto**: due ruoli (`isAdmin`), carrello lato server solo per i clienti, e `POST /api/orders` che è un **ordine finto**. Un agente che vede `paymentLastFour` può mettersi a integrare un pagamento vero.

+ Tutto questo lo sistemate voi: è il laboratorio di questo modulo.

Il ciclo: specifica, piano, esecuzione, verifica

1. **Compito piccolo e chiaro**: “aggiungi al catalogo il filtro per marca”. Non “migliora il catalogo”.
2. **Chiedere il piano** prima: quali file toccherà, quali tipi cambierà, come verificherà. Correggere il piano costa niente; correggere il codice costa.
3. **Lasciarlo lavorare** con `npm run lint` e `npm run build` come vincolo: deve farli passare lui, non voi.
4. **Leggere il diff** come se fosse la PR di un collega junior brillante e un po’ frettoloso.
5. **Provare a mano**: aprite il catalogo, filtrate, provate con la tastiera. L’agente non ha occhi sull’interfaccia.
6. Commit con messaggio vostro. Il prossimo compito parte da un repo pulito.

Un prompt ben fatto: il filtro per marca

```
Aggiungi al catalogo il filtro per marca.

Contesto: GET /api/products accetta GIA' il parametro brand
(backend/src/routes/products.ts, filtro eq(products.brand, brand),
confronto esatto). Il frontend non lo usa: l'interfaccia ProductFilters in
frontend/src/lib/api.ts non ha il campo brand, e CatalogPage.tsx tiene i
filtri in useState. Non esiste un endpoint che elenca le marche.

Requisiti:
- aggiungi brand?: string a ProductFilters e passalo a fetchProducts
- in CatalogPage un Select "Marca" con "Tutte le marche" come default,
  costruito con components/ui/select.tsx (nessuna nuova dipendenza)
- le marche si ricavano dai prodotti ricevuti, ordinate e senza duplicati
- etichette in italiano, coerenti con il resto della pagina
- non toccare gli altri filtri, il backend o components/ui/

Prima dimmi il piano in 5 righe, poi procedi.
Alla fine esegui: cd frontend && npm run lint && npm run build
```

Perché quel prompt funziona

- **Contesto vero**: dice cosa esiste già (il parametro `brand` nel backend) e cosa no (l'endpoint delle marche). Senza, l'agente inventa `GET /api/brands`.
- **Requisiti verificabili**: un campo nel tipo, un componente, un comportamento di default. Si controllano uno per uno nel diff.
- **Vincoli espliciti**: nessuna dipendenza, componenti esistenti, backend intoccato.
- **Criterio di "fatto"**: `npm run lint` e `npm run build` verdi. Farli passare non è lavoro vostro.
- **Il piano prima**: cinque righe da leggere invece di trecento righe da rileggere.
- È la specifica del modulo 5, in piccolo. Lo stesso mestiere.

Secondo esempio: via `window.confirm`

```
In frontend/src/pages/AdminPage.tsx l'eliminazione di un articolo usa
window.confirm. Sostituiscilo con AlertDialog di shadcn/ui.

Il componente non c'e' ancora in src/components/ui/: aggiungilo con
  npx shadcn@latest add alert-dialog
(stile radix-nova, vedi frontend/components.json) e non modificare gli
altri file in components/ui/.

Requisiti:
- titolo e testo in italiano, con il nome dell'articolo da eliminare
- azioni "Annulla" ed "Elimina" (variante distruttiva), chiusura con Esc
- la chiamata deleteProduct e il toast di errore restano come sono
- il dialogo di modifica (ProductFormDialog) non va toccato

Piano in 5 righe, poi procedi. Verifica con npm run lint && npm run build.
```

Un compito con un confine netto: un file, un componente aggiunto, nessun cambio di logica.

Revisione del diff: cosa cercare su questo repo

- File toccati che **non c'entrano** con il compito
- `any`, `@ts-ignore`, regole di lint disattivate
- Tipi aggiornati in `lib/api.ts`? Un filtro nuovo è un campo nuovo in `ProductFilters`
- **Dipendenze** nuove in `package.json`: perché?
- Chiamate HTTP scritte nella pagina invece che in `lib/api.ts`
- Accessibilità dei controlli nuovi: label, `aria-label`, tastiera
- Segreti, URL assoluti al posto del proxy, `console.log`
- Messaggi in italiano coerenti con il resto dell'interfaccia
- I quattro stati della schermata ancora tutti gestiti
- Lo capite abbastanza da spiegarlo a un collega?

Se no

Chiedete all'agente di spiegare, semplificare o rifare. "Non capisco" è un motivo sufficiente per rifiutare un diff.

Strumenti collegati: MCP e automazione del browser

- **MCP** (Model Context Protocol): standard per collegare all'agente strumenti esterni: il browser, il database, Figma, l'issue tracker, la documentazione di una libreria.
- Con un server MCP per il browser (Playwright) l'agente **apre minicommerce, filtra il catalogo, fa screenshot** e verifica da solo che il filtro marca funzioni.
- Con un MCP su SQLite legge le categorie e gli articoli del seed invece di indovinare i nomi.
- Con la documentazione ufficiale (Context7 e simili): meno API inventate, meno Tailwind v3 in un progetto v4.
- Regola: ogni strumento in più è potere in più. Lettura prima della scrittura, e mai le credenziali di produzione.

Le competenze che valgono di più nel 2026

- **Leggere codice** velocemente e con giudizio: è l'attività principale.
- **Specificare**: trasformare "il catalogo è lento" in "paginare `GET /api/products` e smettere di rispedire le immagini base64 in ogni risposta".
- **Architettura e confini**: decidere dove stanno le cose (lo avete fatto in ogni modulo).
- **Giudizio di prodotto e UX**: l'agente non sa se un filtro in più aiuta o confonde.
- **Debug sistematico**: ipotesi, esperimento, conferma. L'agente accelera, voi guidate.
- **Fondamenti**: HTTP, browser, rendering, tipi. Senza, non riconoscete i suoi errori.

Detto altrimenti

L'agente moltiplica ciò che sapete. Moltiplicare zero dà zero.

Laboratorio 8

Laboratorio: Migliorare l'`AGENTS.md` del vostro repository

1. Riscrivete `AGENTS.md`: le scelte dei moduli 1–3, i quattro stati, le regole di accessibilità, la regola delle virgolette allineata al codice vero, le parti nuove del progetto (ruoli, carrello, ordine finto) e una sezione "cosa non fare".
2. Dategli poi un compito vero: il filtro per marca, oppure `AlertDialog` al posto di `window.confirm`.
3. Tenete un **diario**: prompt, piano proposto, cosa avete corretto e perché, tempo impiegato.
4. Revisione incrociata: un compagno rivede il vostro diff senza sapere cosa ha scritto l'agente e cosa voi.
5. Rifate lo stesso compito **senza** il file di contesto e confrontate i due diff.

Domande di verifica

Perché il piano prima del codice? Tre segnali di un diff da rifiutare? Quale riga dell'`AGENTS.md` attuale vi ha salvato il database locale?